Application Deployment and troubleshooting – Day 2 Ops (OpenShift)

1. Monitoring & Observability
   - Pod status and health checks

```
oc get pod
oc describe pod <pod-name>
```

livenessProbe:
  httpGet:
    path: /health
    port: 8080

- Logs and events

```
oc logs <pod-name>
oc logs <pod-name> -c <container-name>
oc logs <pod-name> --previous
oc logs -f <pod-name>
oc get events
oc get events --sort-by=.metadata.creationTimestamp
```

- Metrics (CPU, memory)

```
oc adm top pods
oc adm top nodes
```

Shows:

- CPU usage
- Memory usage

OpenShift Web Console

Go to:

- **Observe → Metrics / Dashboards**

You'll see:

- CPU graphs
- Memory usage
- Pod performance over time

- Alerts and dashboards

Alerts

In Web Console:

- Go to **Observe → Alerting**
  You can:
- See firing alerts
- Check severity (warning/critical)
- View alert rules

---

2. Troubleshooting & Debugging

- ***Pod failures (CrashLoopBackOff, Pending, etc.)***
  `oc get pods`
  Common states & meaning:
- **CrashLoopBackOff**
  App is crashing repeatedly
  Causes:
  - Wrong config
  - App error
  - Missing dependency
- **Pending**
  Pod not scheduled
  Causes:
  - No resources (CPU/memory)
  - PVC not bound
  - Node issue
- **ImagePullBackOff**
  Image cannot be pulled
  Causes:
  - Wrong image name
  - No access to registry
    `oc describe pod <pod-name>`
    Look at:
- Events
- Errors like:
  - `FailedScheduling`
  - `FailedMount`
  - `ErrImagePull`

- Log analysis
  `oc log <pod-name>`
  multiple container
  `oc log <pod-name> -c <container-name>`

For crashed container

```
oc log <pod-name> --previous
```

- Describe resources

  pod

  ```
  oc describe pod <pod-name>
  ```

  deployment

  ```
  oc describe deployment <deployment-name>
  ```

  service

  ```
  oc describe service <service-name>
  ```

What to check:

- Events section (bottom)
- Resource limits
- Environment variables
- Mounted volumes

- Exec into containers

  Used when:
- App is running but not behaving correctly
- You want to inspect inside container

  ```
  oc exec -it <pod-name> -- /bin/bash
  ```

  if bash not available

  ```
  oc exec -it <pod-name> -- /bin/sh
  ```

What you can do inside:

- Check files:

  ```
  ls /app
  ```
- Check env variables:

  ```
  env
  ```
- Test connectivity:

  ```
  curl <service>
  ```
- Check running processes:

  ```
  ps aux
  ```

Real Debug Flow (IMPORTANT)

1. oc get pods
2. oc describe pod
3. oc logs
4. oc logs --previous (if crash)
5. oc exec (if needed)

---

3. **Scaling & Performance**

- Manual scaling
  `oc scale deployment <app-name> --replicas=N`

- Auto scaling (HPA)
  HPA – Horizontal Pod Autoscaler
  `oc autoscal deployment <app-name> --min=N --max=M --cpu-persent=P`
  `oc get hpa`
  `oc describe hpa`
  How it works:
- Monitors metrics (via Prometheus)
- Adds/removes pods automatically

- Resource requests and limits tuning
  What it is:
  You define how much CPU/memory a container:
- **requests** (minimum guaranteed)
- **limits** (maximum allowed)

```
resources:
  requests:
    memory: "256Mi"
    cpu: "200m"
  limits:
    memory: "512Mi"
    cpu: "500m"
```

Check current usage:
`oc adm top pods`

---

4. Updates & Rollouts

- Rolling updates

  What it is:

  Update your application **gradually**, without downtime.

  Old pods are terminated **step by step** while new ones are created.

  Option 1: Update image

  ```
  oc set image deployment/myapp mycontainer=myimage:v2
  ```

  Option 2: Edit deployment

  ```
  oc edit deployment myapp
  ```

  status

  ```
  oc rollout status deployment/myapp
  ```

  history

  ```
  oc rollout history deployment/myapp
  ```

- Rollbacks

  What it is:

  Revert to **previous working version** if something breaks.

  ```
  oc rollout undo deployment/myapp
  ```

  rollback to specific version

  ```
  oc rollout undo deployment/myapp --to-revision=2
  ```

- Image updates and triggers

  What it is:

  Automatically redeploy app when:

- New image is available
- Config changes

  ImageStream (OpenShift feature)

- Tracks image versions
- Triggers deployment when image changes

  Types of triggers:

  1. Image Change Trigger

  When new image pushed → auto deploy

  example:

  ```
  oc set triggers deployment/myapp --from-image=myimage:latest -c
  <container-name>
  ```

  Check triggers

  ```
  oc set triggers deployment/myapp --list
  ```

after deployment of new image app breakes

```
oc rollout status deployment/myapp
```

```
oc logs <pod>
oc rollout undo deployment/myapp
```

---

5. Configuration Management

- ConfigMaps updates

  ConfigMap stores non-sensitive configuration data (like environment variables, app configs, or properties files)

  Decouples (separate) config from application code

  create configmap

  ```
  oc create configmap <name-of-config> --from-literal=LOG_LEVEL=info
  ```

  OR

  ```
  oc create configmap <name-of-config> --from-file=<file.properties>
  ```

  update configmap

  ```
  oc edit configmap <config-name>
  ```

  apply to deployment

  mount configmap as:

- environment variable

envFrom:
- configMapRef:
name: myconfig

- volume

volumes:
- name: config-volume
configMap:
name: myconfig

Trigger redeployment after update

```
oc rollout restart deployment/myapp
```

- Secrets management

  Secret stores sensitive information:

- Passwords

  Create secret

```
oc create secret generic <name-of-secret> --from-literal=DB_PASSWORD=<passwork>
```

or from file

```
oc create secret generic <name-of-secret> --from-file=ssh-key=id_rsa
```

Use Secret in Deployment

As **environment variable**:

```
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: mysecret
      key: DB_PASSWORD
```

As **volume**:

```
volumes:
- name: secret-volume
  secret:
    secretName: mysecret
```

Updating Secrets

```
oc edit secret mysecret
```

redeploy

```
oc rollout restart deployment/myapp
```

- Tokens

  Purpose:
- Used for **authentication and authorization** (e.g., service accounts, API access)
- Example: GitHub token, OAuth token, API keys

  create token

  ```
  oc create secret generic <TOKEN> --from-litteral=API_TOKEN=<TOKEN>
  ```
- Certificates

  Used for **TLS encryption, secure communication**, or client auth

  Example: SSL certificates for HTTPS, or client certs for API access

  ```
  oc create secret tls mycert --cert=server.crt --key=server.key
  ```

Updating Tokens or Certificates

1. Update secret:
   ```
   oc create secret generic mytoken --from-literal=API_TOKEN=newvalue -o yaml --dry-run=client | oc apply -f -
   oc rollout restart deployment/myapp
   ```

- Redeployment on config change
  Ways to trigger redeployment automatically:

1. **DeploymentConfig triggers** (OpenShift-specific)
   - Can watch ConfigMaps/Secrets changes
2. **Manual rollout**
   ```
   oc rollout restart deployment/myapp
   ```

---

6. Storage & Data Management
   - PVC monitoring
     What it is:

- **PVC** = request for storage by a pod
- Bound to a **PV (PersistentVolume)**
  ```
  oc get pvc
  ```
  bound >> working
  ```
  oc describe pvc <pvc-name>
  ```

To check status
```
oc adm top pvc
```
inside pod
```
oc exec -it <pod-name> --df -Th
```

- Volume expansion
  Increase storage size **without deleting data**
  Requirement:
- StorageClass must support expansion
  ```
  oc edit pvc <pvc-name>
  ```
  edit:

```
resources:
  requests:
    storage: 20Gi   # increase from 10Gi
```

```
oc get pvc
```

check events:

```
oc describe pvc <pvc-name>
```

- Backup and restore
  Protect data from:
- Failures
- Deletion
- Corruption
  Backup Methods

1. Application-level backup (BEST for DBs)

```
#mysql
mysqldump -u user -p db > backup.sql
```

2. Volume-level backup

- Snapshot (cloud storage)
- Storage provider tools

3. OpenShift resources backup

```
oc get all -o yaml > backup.yaml
```

Restore DB

```
mysql -u user -p db < backup.sql
```

Restore PVC data

- Reattach volume
- Restore snapshot

---

7. Networking & Access

- Services troubleshooting
  Services Troubleshooting
  What a Service does:
- Exposes pods **inside the cluster**
- Provides **stable IP + DNS**

```
oc get svc
oc describe svc <svc-name>
```
--> pod port selector

```
oc get endpoints <service-name>
oc get pods --show-labels
```

- Routes and external access
  What a Route does:
- Exposes service **outside the cluster**
- OpenShift-specific
  ```
  oc get routes
  oc describe routes <route-name>
  ```
  test:
  ```
  curl http://<route-name>
  ```
  Debug flow:
  Route --> Service --> Pod

- DNS and connectivity issues
  Check DNS inside cluster
  ```
  oc exec -it <pod-name> -- nslookup <service-name>
  ```
  Test connectivity
  ```
  oc exec -it <pod-name> -- curl http://<service-name>:<port>
  ```
  Check pod-to-pod communication
  ```
  oc exec -it <pod-name> -- ping <other-pod-ip>
  ```
  Check network policies
  ```
  oc get networkpolicy
  ```
  Common issues:
- DNS not resolving
- NetworkPolicy blocking traffic
- Port not exposed
- Firewall/security rules
  ```
  Debug Flow
  ```

App not accessible
↓
Check Route (oc get routes)
↓
Check Service (oc get svc)
↓
Check Endpoints (oc get endpoints)
↓
Check Pods (oc get pods)
↓
Test inside pod (curl/nslookup)

---

8. Security & Access Control

- RBAC roles and bindings
  Controls **who can do what** in the cluster.
  Check roles

oc get roles
oc get clusterroles

Check bindings

oc get rolebindings
oc get clusterrolebindings

Create role

```
oc create role pod-reader --verb=get,list --resource=pods
```

Bind role to user

```
oc create rolebinding read-pods \
  --role=pod-reader \
  --user=user1
```

Verify access

```
oc auth can-i get pods --as=user1
```

- Service accounts

- SCC policies
- Secret rotation

8. Resource Optimization
   - CPU/memory usage analysis
   - Quotas and limits
   - Cost optimization
9. Automation & CI/CD

- Pipeline-triggered deployments
- GitOps workflows
- Automation of scaling and updates